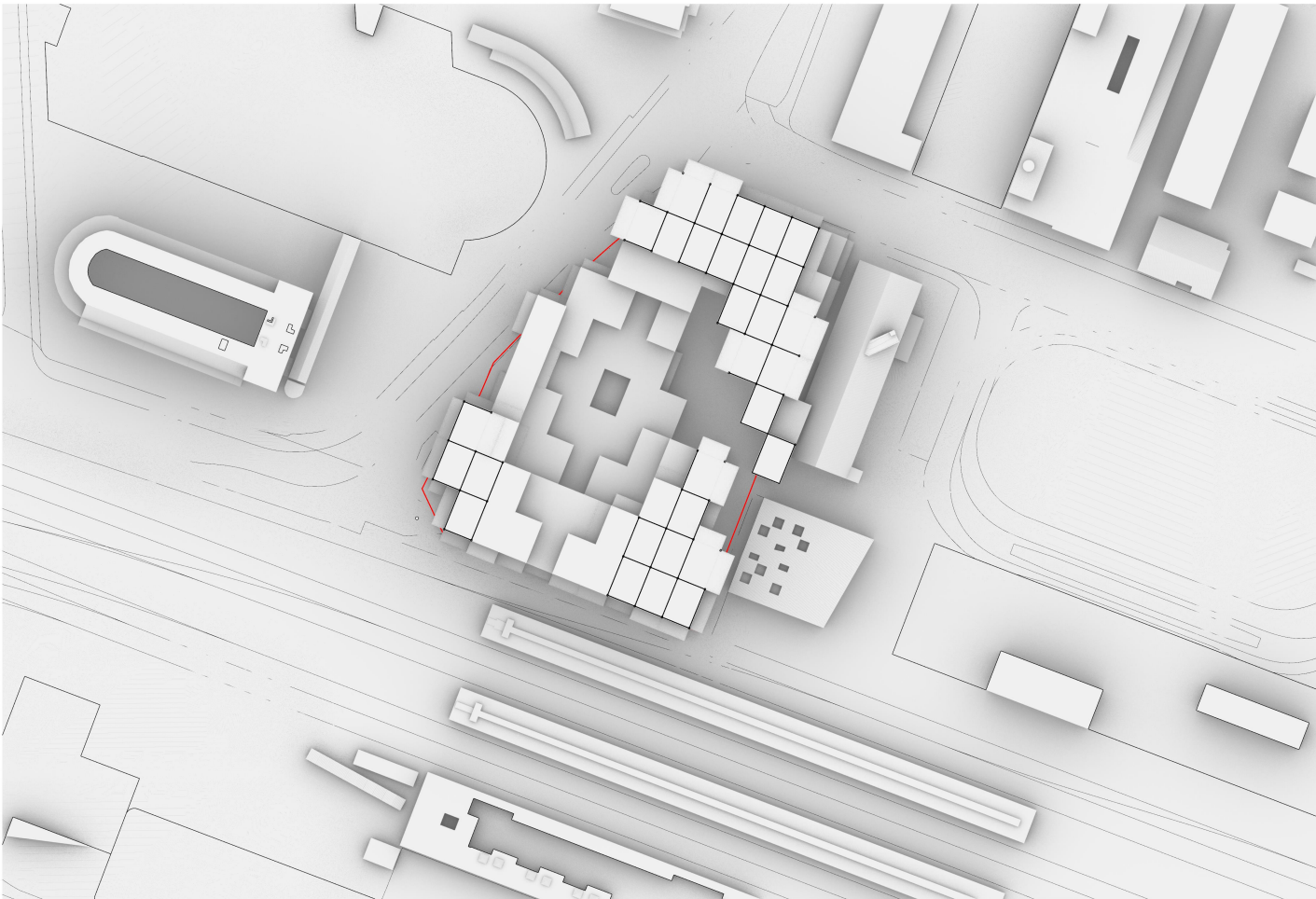


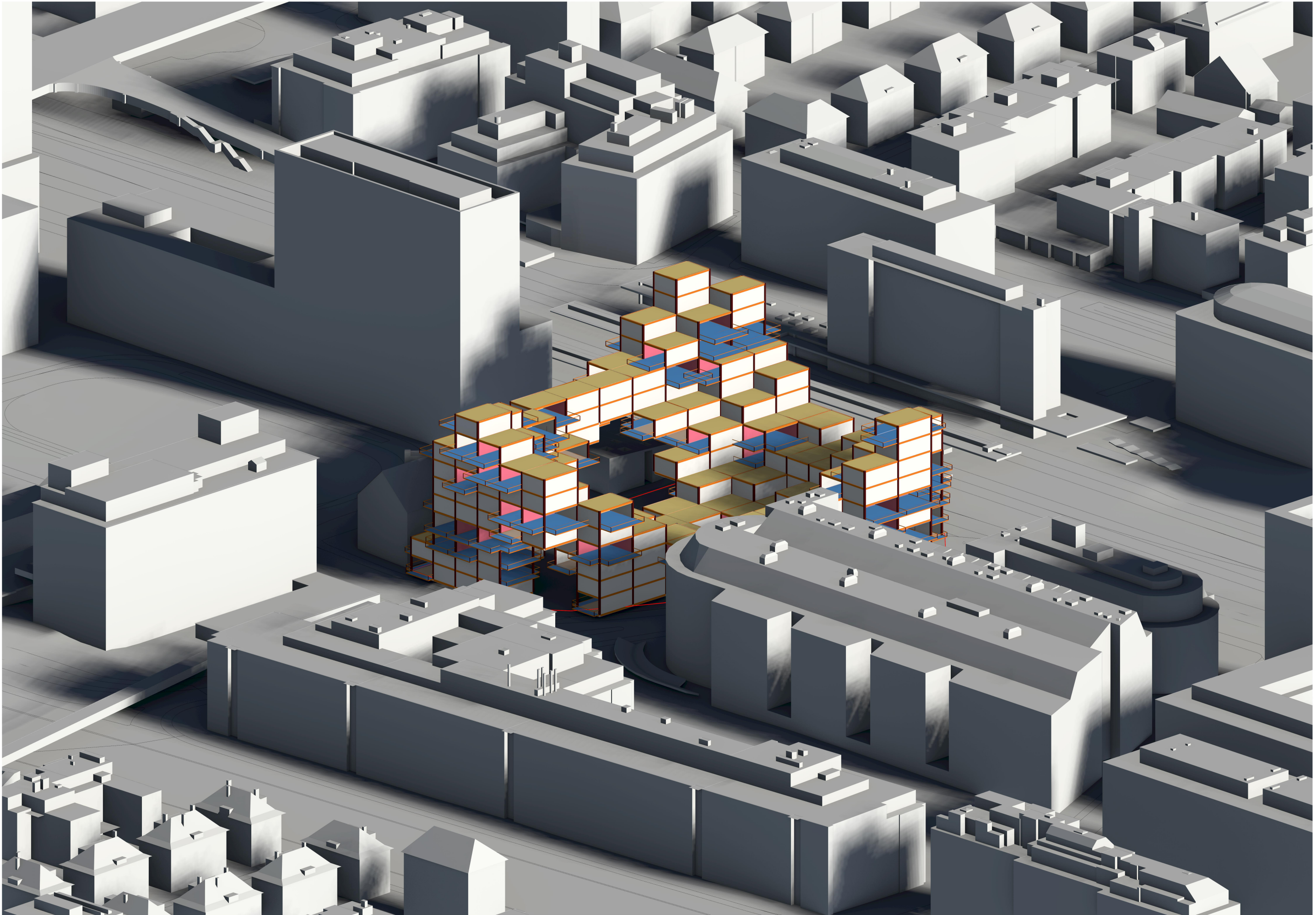


A parabola filter is used to create an open space in the middle of the complex to introduce sunlight. An additive Brep filter constraints the geometry within the site, and a subtractive Brep filter opens the connection from lower right to top left of the plan.

Internal walls are added to separate internal space.
In this building, The volume of columns are 354.21 m³, beams 1254 m³, internal walls 2624 m³, facade 1112 m³ and slabs 1112 m³.

CODING ARCHITECTURE I / HS24

Wang, Yiqiao



```

131 @staticmethod
132 def get_halfface_lower_two_points(halfface_idx, volmesh):
133     """
134     The Function identifies the outside of halfface, and returns a tuple of two points,
135     p0 and p1. This can be used to
136     Locate the balcony on the facade.
137
138     return: return the Lower left and Lower right points of the halfface
139     rtype: tuple of Point
140
141     # check if halfface is vertical
142     normal = volmesh.halfface_normal(halfface_idx).inverted()
143
144     points = volmesh.face_points(halfface_idx)
145     # find the two points with the lowest z value
146     lower_points = []
147     for point in points:
148         if len(lower_points) < 2:
149             lower_point.append(point)
150         else:
151             if point.z < lower_points[0].z:
152                 lower_points[0] = point
153             elif point.z < lower_points[1].z:
154                 lower_point[1] = point
155
156     # also make sure that the normal of the halfface cross product
157     # with the vector of the two points is pointing upwards
158     # so that (normal, [points[0] -> points[1], zero]) is a right-hand coordinate system
159     if not np.cross(lower_points[1] - lower_points[0], det(vector, Axis(0)) > 0:
160         lower_points[0], lower_point[1] = lower_point[1], lower_points[0]
161
162     return lower_point[0], lower_point[1]
163
164 def get_balcony_points(self, p0, p1, length, height):
165     """
166     Calculate the points of a balcony given two initial points, length, and height.
167
168     :param p0: The starting point of the balcony (Lower left corner).
169     :type p0: Point
170     :param p1: The ending point of the balcony (Lower right corner).
171     :type p1: Point
172     :param length: The length of the cantilever of the balcony.
173     :type length: float
174     :param height: The height of the balcony.
175     :type height: float
176     return: A list of 8 points representing the balcony.
177     rtype: list of Point
178     """
179
180     vv = Vector.from_start_end(p0, p1)
181     t_x = vv.cross(Vector, Axis(1)).utilized()
182     t_z = Translation.from_vector(vv * length)
183     t_x = Translation.from_vector(Vector, Axis(0) * height)
184     p0 = p0.transformed(t_x)
185     p1 = p0.transformed(t_x)
186     p0 = p0.transformed(t_z)
187     p1 = p0.transformed(t_z)
188     p2 = p1.transformed(t_x)
189     p3 = p1.transformed(t_z)
190     p4 = p2.transformed(t_x)
191     p5 = p2.transformed(t_z)
192     return [p0, p1, p2, p3, p4, p5, p6, p7]

```